



Dwight Look College of

ENGINEERING
TEXAS A&M UNIVERSITY

Team 56: Openroad VLSI design 2 Status Update 1

Kevin, Talha, Githin, Giovani

Sponsor: Kevin Nowka

TA: Michael Johnson-Moore

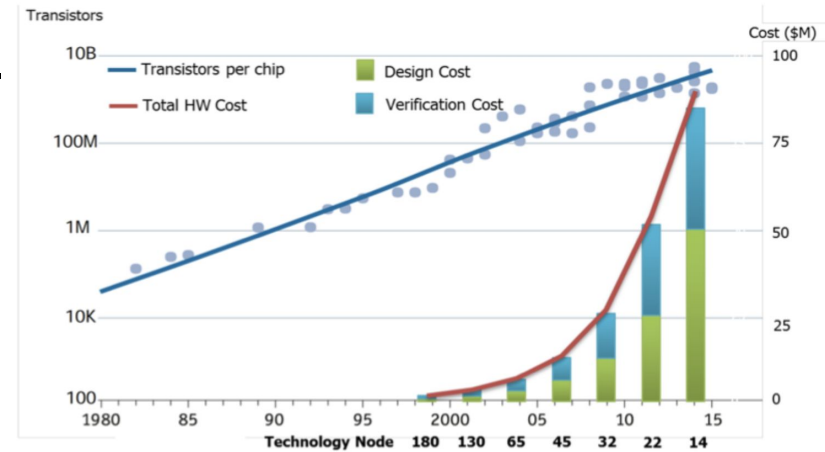
Project Summary

Problem statement:

- Traditional RTL-to-GDSII chip design is expensive, creating a cost barrier to developing a System on a Chip (SoC).

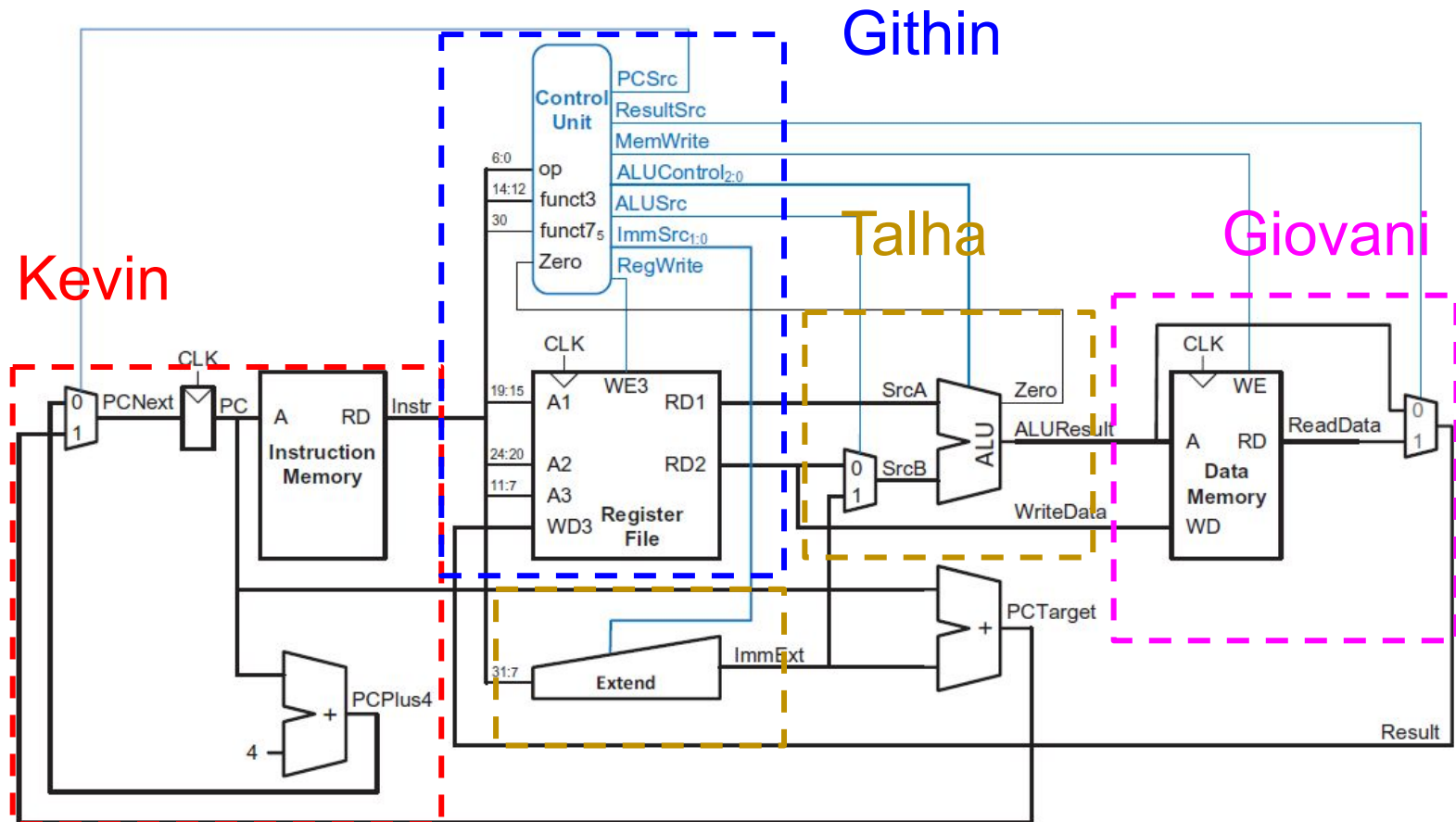
Our Solution:

- Create a 16-bit RISC-V using OpenROAD, which automates chip design by only needing only an RTL description to generating GDSII, significantly reducing cost.



OpenROAD/OpenLane

System Overview





Dwight Look College of

ENGINEERING
TEXAS A&M UNIVERSITY

Major Project Changes for 404

No Major Changes



Project Timeline

- Finalized individual subsystems last week (Jan. 26)
- Start integrating/validating this week (Feb. 2)
- Implement pipeline stages (Feb. 16)
- Validate chip at each stage (Mar. 2)
- Start OpenLane/OpenROAD for pipelined processor (Mar. 23)

Fetch

Kevin Umanzor

Accomplishments since 403 15 hrs of effort	Ongoing progress/problems and plans until next presentation
- Finished creating the 16-bit PC counter RTL file and its corresponding testbench. - Started creating the top-level RISC-V Core module for the CPU.	- Determine how many I/O pins are required and what external interfaces we will actually support. - Research and compare the fabrication paths between Tiny Tapeout and Chip Ignite to decide which is better for this project.



Fetch

Kevin Umanzor

```
+-----+-----+-----+-----+
|                                     |
|               Test 6: JALR to 0x0100 |
|-----+-----+-----+-----+
| Time | PC | Operation | Status |
|-----+-----+-----+-----+
| Current PC before JALR: 0x0032 |
|-----+-----+-----+-----+
| 286 | 0x0100 | JALR to 0x0100 | |
| 296 | 0x0102 | Sequential | |
| 306 | 0x0104 | Sequential | |
| 316 | 0x0106 | Sequential | |
|-----+-----+-----+-----+

+-----+-----+-----+-----+
|                                     |
|               ALR Unaligned (0x0055 -> 0x0054) |
|-----+-----+-----+-----+
| Time | PC | Operation | Status |
|-----+-----+-----+-----+
| Current PC before JALR: 0x0106 |
|-----+-----+-----+-----+
| 326 | 0x0054 | ligned correctly | PASS |
| 336 | 0x0056 | Sequential | |
| 346 | 0x0058 | Sequential | |
|-----+-----+-----+-----+

+-----+-----+-----+-----+
|                                     |
|               : Priority (JALR > JAL > Branch) |
|-----+-----+-----+-----+
| Time | PC | Operation | Status |
|-----+-----+-----+-----+
| 486 | 0x0200 | priority correct | PASS |
|-----+-----+-----+-----+

+-----+-----+-----+-----+
| PC Counter Test Complete |
+-----+-----+-----+-----+
```

```
TEST 1: Sequential Memory Read
+-----+-----+-----+-----+
| PC (hex) | Word Idx | Instruction | Status |
+-----+-----+-----+-----+
| 0000 | 0000 | 1234 | OK |
| 0002 | 0001 | 5678 | OK |
| 0004 | 0002 | 9abc | OK |
| 0006 | 0003 | def0 | OK |
| 0008 | 0004 | aaaa | OK |
| 000a | 0005 | 5555 | OK |
| 000c | 0006 | ffff | OK |
| 000e | 0007 | 0000 | OK |
| 0010 | 0008 | 1111 | OK |
| 0012 | 0009 | 2222 | OK |
| 0014 | 000a | 3333 | OK |
| 0016 | 000b | 4444 | OK |
| 0018 | 000c | beef | OK |
| 001a | 000d | cafe | OK |
| 001c | 000e | dead | OK |
| 001e | 000f | c0de | OK |
+-----+-----+-----+-----+
RESULT: PASS (16/16 reads successful)
```

```
TEST 5: First 8 Instructions
+-----+-----+-----+-----+
| Address | Instruction | Status |
+-----+-----+-----+-----+
| 0000 | 1234 | OK |
| 0002 | 5678 | OK |
| 0004 | 9abc | OK |
| 0006 | def0 | OK |
| 0008 | aaaa | OK |
| 000a | 5555 | OK |
| 000c | ffff | OK |
| 000e | 0000 | OK |
+-----+-----+-----+-----+
RESULT: PASS (8/8 sequential reads)

TEST 6: Out-of-Range Access
[PASS] PC=0x0200 (>255 words) -> Instr=0x0000 (bounds check works)
[PASS] PC=0x01fe (max valid) -> Instr=0x0000
RESULT: PASS (2/2 boundary tests)

=====
FINAL TEST SUMMARY
=====

Total Tests: 33
Passed: 33
Failed: 0

*** ALL TESTS PASSED ***
Status: SUCCESS
=====
```

Decode

Githin Johny

Accomplishments since 403 18 hrs of effort	Ongoing progress/problems and plans until next presentation
- Completed expansion of testbench for coverage - Completed 100% test coverage of all cases	- Research pipeline implementation for the Decode subsystem - Identify interfaces between connecting subsystems



Decode - Control Unit

Githin Johny

Time	Opcode	Func	Zero	PCSrc	ResultSrc	MemWrite	ALUControl	ALUSrc	ImmSrc	RegWrite	Instruction
6000	000	0000	0	0	0	0	0000	0	00	1	ADD
Result: PASSED											
16000	000	0001	0	0	0	0	0001	0	00	1	SUB
Result: PASSED											
26000	000	0010	0	0	0	0	0010	0	00	1	XOR
Result: PASSED											
36000	000	0011	0	0	0	0	0011	0	00	1	OR
Result: PASSED											
46000	000	0100	0	0	0	0	0100	0	00	1	AND
Result: PASSED											
56000	000	0101	0	0	0	0	0101	0	00	1	SLL
Result: PASSED											
66000	001	0000	0	0	0	0	0000	1	00	1	ADDI
Result: PASSED											
76000	001	0001	0	0	0	0	0010	1	00	1	XORI
Result: PASSED											
86000	001	0010	0	0	0	0	0011	1	00	1	ORI
Result: PASSED											
96000	001	0011	0	0	0	0	0100	1	00	1	ANDI
Result: PASSED											
106000	001	1101	0	1	0	0	0000	1	00	1	JALR
Result: PASSED											
116000	010	0000	0	0	1	0	0000	1	00	1	LW
Result: PASSED											
126000	011	0000	0	0	0	1	0000	1	00	0	SW
Result: PASSED											
136000	100	0000	1	1	0	0	0001	0	01	0	BEQ (taken)
Result: PASSED											
146000	100	0000	0	0	0	0	0001	0	01	0	BEQ (not taken)
Result: PASSED											
156000	100	0001	1	0	0	0	0001	0	01	0	BNE (not taken)
Result: PASSED											
166000	100	0001	0	1	0	0	0001	0	01	0	BNE (taken)
Result: PASSED											
176000	101	0000	0	1	0	0	0000	1	10	1	JAL
Result: PASSED											

Control Unit coverage results



Decode - Register File

Githin Johny

Test#	Operation	A3	WD3	RegWrite	A1	A2	Expected RD1	Expected RD2	Pass?
1	Write X1	001	100	1	001	010	100	0	YES
2	Write X2	010	200	1	001	010	100	200	YES
3	Write X0	000	999	1	000	001	0	100	YES
4	No write	011	300	0	011	010	0	200	YES
5	Overwrite X1	001	111	1	001	010	111	200	YES
6	Read X0	-	-	0	000	000	0	0	YES
7	Simultaneous R/W	010	555	1	010	001	200	111	YES
8	Read X2 after write	-	-	0	010	001	555	111	YES

Register File Test Bench Completed.

Register File coverage results



Execute

Talha Ibrahim

Accomplishments since 403 16 hrs of effort	Ongoing progress/problems and plans until the next presentation
Completed further implementing test cases for all ALU/Sign Extender operations	- Start connecting dependent subcomponents - Research pipeline implementation for chip



Execute

Talha Ibrahim

ALU coverage results

Instr	SrcA	SrcB	ALU Ctrl	Expected	ALU Result	Zero	Zero Correct?	Pass?
ADD	10	5	0000	15	15	0	YES	YES
ADD_W	ffff	0001	0000	0000	0000	1	YES	YES
ADD_SO	7fff	0001	0000	8000	8000	0	YES	YES
ADD_NO	8000	8000	0000	0000	0000	1	YES	YES
ADD_X	8000	7fff	0000	ffff	ffff	0	YES	YES
SUB	000a	0005	0001	0005	0005	0	YES	YES
SUB0	000f	000f	0001	0000	0000	1	YES	YES
SUB_UF	0000	0001	0001	ffff	ffff	0	YES	YES
SUB_L	8000	7fff	0001	0001	0001	0	YES	YES
SUB_M0	ffff	0000	0001	ffff	ffff	0	YES	YES
SUB_0M	0000	ffff	0001	0001	0001	0	YES	YES
XOR	000a	0005	0010	000f	000f	0	YES	YES
XOR_Z	1234	0000	0010	1234	1234	0	YES	YES
XOR_F	aaaa	ffff	0010	5555	5555	0	YES	YES
XOR_0	55aa	55aa	0010	0000	0000	1	YES	YES
XOR_L	ffff	8000	0010	7fff	7fff	0	YES	YES
XOR_X	aaaa	5555	0010	ffff	ffff	0	YES	YES
OR0	0000	0000	0011	0000	0000	1	YES	YES
OR_PASS	0000	ffff	0011	ffff	ffff	0	YES	YES
OR_SAT	ffff	ffff	0011	ffff	ffff	0	YES	YES
OR_ALT	aaaa	5555	0011	ffff	ffff	0	YES	YES
OR_MSB	8000	0000	0011	8000	8000	0	YES	YES
OR_LSB	0001	0000	0011	0001	0001	0	YES	YES
AND0	0000	0000	0100	0000	0000	1	YES	YES
AND_ID	ffff	ffff	0100	ffff	ffff	0	YES	YES
AND_Z	ffff	0000	0100	0000	0000	1	YES	YES
AND_S	a5a5	a5a5	0100	a5a5	a5a5	0	YES	YES
AND_A	aaaa	5555	0100	0000	0000	1	YES	YES
AND_M	8000	ffff	0100	8000	8000	0	YES	YES
AND_L	0001	ffff	0100	0001	0001	0	YES	YES
SLL0	1234	0000	0101	1234	1234	0	YES	YES
SLL1	0001	0001	0101	0002	0002	0	YES	YES
SLL_M	0001	000f	0101	8000	8000	0	YES	YES
SLL_OV	8000	0001	0101	0000	0000	1	YES	YES
SLL15	00ff	000f	0101	8000	8000	0	YES	YES
SLL_Z	0000	0007	0101	0000	0000	1	YES	YES
SLL_L	0003	0004	0101	0030	0030	0	YES	YES
SRL	49167	4	0110	3072	3072	0	YES	YES
SRL	12345	0	0110	12345	12345	0	YES	YES
SRL	32769	1	0110	16384	16384	0	YES	YES
SRL	1	4	0110	0	0	1	YES	YES
SRL	0	7	0110	0	0	1	YES	YES
SRL	65535	4	0110	4095	4095	0	YES	YES
SRL	32768	15	0110	1	1	0	YES	YES
SRA0	0	4	0111	0	0	1	YES	YES
SRA_MSB	-32768	4	0111	-2048	-2048	0	YES	YES
SRA_LSB	1	1	0111	0	0	1	YES	YES
SRA_NEG	-1	4	0111	-1	-1	0	YES	YES
SRA_ALT	-21846	1	0111	-10923	-10923	0	YES	YES
JAL	100	-	1000	102	102	0	YES	YES
JAL_ZERO	0	-	1000	2	2	0	YES	YES
JAL_MAX	ffff	-	1000	0001	0001	0	YES	YES
JAL_EDGE	7fff	-	1000	8001	8001	0	YES	YES

Sign Extender coverage results

Instr	ImmSrc	Type	ImmExt	Expected	Pass
0000000000000000	000	R-TYPE	0000000000000000	0000000000000000	1
1111111111111111	000	R-TYPE	0000000000000000	0000000000000000	1
1010101010101010	000	R-TYPE	0000000000000000	0000000000000000	1
0101010101010101	000	R-TYPE	0000000000000000	0000000000000000	1
0000111111111111	000	R-TYPE	0000000000000000	0000000000000000	1
1000000000000000	000	R-TYPE	0000000000000000	0000000000000000	1
0000000000000000	001	I-TYPE	0000000000000000	0000000000000000	1
0010000000000000	001	I-TYPE	0000000000000001	0000000000000001	1
0100000000000000	001	I-TYPE	0000000000000010	0000000000000010	1
0110000000000000	001	I-TYPE	0000000000000011	0000000000000011	1
1000000000000000	001	I-TYPE	1111111111111100	1111111111111100	1
1010000000000000	001	I-TYPE	1111111111111101	1111111111111101	1
1100000000000000	001	I-TYPE	1111111111111110	1111111111111110	1
1110000000000000	001	I-TYPE	1111111111111111	1111111111111111	1
0111110000000000	010	S-TYPE	0000000001111111	0000000001111111	1
1000000000000000	010	S-TYPE	1111111111000000	1111111111000000	1
0000001000000000	010	S-TYPE	0000000000000001	0000000000000001	1
1000001000000000	010	S-TYPE	1111111111000001	1111111111000001	1
0010101000000000	010	S-TYPE	0000000000010101	0000000000010101	1
0111110000000000	011	B-TYPE	0000000001111110	0000000001111110	1
0000001000000000	011	B-TYPE	0000000000000001	0000000000000001	1
0010101000000000	011	B-TYPE	0000000000101010	0000000000101010	1
1000000000000000	011	B-TYPE	1111111110000000	1111111110000000	1
1000001000000000	011	B-TYPE	1111111110000001	1111111110000001	1
1010101000000000	011	B-TYPE	1111111101010101	1111111101010101	1
0000000000000000	100	U-TYPE	0000000000000000	0000000000000000	1
0000000011111000	100	U-TYPE	0000000011111000	0000000011111000	1
0111111111111000	100	U-TYPE	0111111111111000	0111111111111000	1
1000000000000000	100	U-TYPE	1000000000000000	1000000000000000	1
1111111111111000	100	U-TYPE	1111111111111000	1111111111111000	1
0000000000000000	101	J-TYPE	0000000000000000	0000000000000000	1
0000001111111111	101	J-TYPE	0000001111100000	0000001111100000	1
1000000000000000	101	J-TYPE	1000000000000000	1000000000000000	1
1111111111111111	101	J-TYPE	1111111111000000	1111111111000000	1
0001000000000000	101	J-TYPE	0001000000000000	0001000000000000	1
1010101010101010	101	J-TYPE	1010101010000000	1010101010000000	1
1111111111111111	110	X	0000000000000000	0000000000000000	1



Data Memory

Giovani Giagnacovo

Accomplishments since 403 12 hrs of effort	Ongoing progress/problems and plans until the next presentation
Expanded test bench to test all the 8 16 bit memory addresses, as opposed to just a few.	- Finish validating 16 byte SRAM and 2x1 Mux components. - Integrate memory components with other subsystems in complete CPU module



Data Memory

Giovani Giagnacovo

```
=====
TESTING ADDRESS 0
=====
OK  W-only: addr=0 output=0000
OK  R-only: addr=0 data=0x1000
OK  RW-both (bypass NOW): addr=0 = 0x2000
OK  RW-both (read-back): addr=0 = 0x2000
=====

TESTING ADDRESS 1
=====
OK  W-only: addr=1 output=0000
OK  R-only: addr=1 data=0x1001
OK  RW-both (bypass NOW): addr=1 = 0x2001
OK  RW-both (read-back): addr=1 = 0x2001
=====

TESTING ADDRESS 2
=====
OK  W-only: addr=2 output=0000
OK  R-only: addr=2 data=0x1002
OK  RW-both (bypass NOW): addr=2 = 0x2002
OK  RW-both (read-back): addr=2 = 0x2002
=====

TESTING ADDRESS 3
=====
OK  W-only: addr=3 output=0000
OK  R-only: addr=3 data=0x1003
OK  RW-both (bypass NOW): addr=3 = 0x2003
OK  RW-both (read-back): addr=3 = 0x2003
=====
```

```
=====
TESTING ADDRESS 4
=====
OK  W-only: addr=4 output=0000
OK  R-only: addr=4 data=0x1004
OK  RW-both (bypass NOW): addr=4 = 0x2004
OK  RW-both (read-back): addr=4 = 0x2004
=====

TESTING ADDRESS 5
=====
OK  W-only: addr=5 output=0000
OK  R-only: addr=5 data=0x1005
OK  RW-both (bypass NOW): addr=5 = 0x2005
OK  RW-both (read-back): addr=5 = 0x2005
=====

TESTING ADDRESS 6
=====
OK  W-only: addr=6 output=0000
OK  R-only: addr=6 data=0x1006
OK  RW-both (bypass NOW): addr=6 = 0x2006
OK  RW-both (read-back): addr=6 = 0x2006
=====

TESTING ADDRESS 7
=====
OK  W-only: addr=7 output=0000
OK  R-only: addr=7 data=0x1007
OK  RW-both (bypass NOW): addr=7 = 0x2007
OK  RW-both (read-back): addr=7 = 0x2007
=====
Data Memory test complete.
```



Dwight Look College of

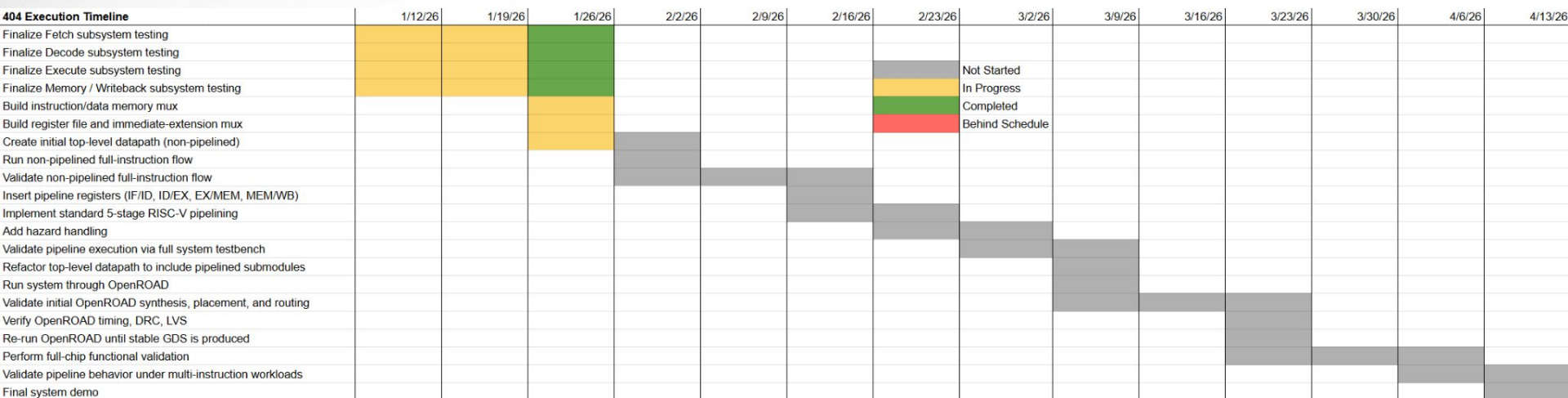
ENGINEERING
TEXAS A&M UNIVERSITY

Parts Ordering Status

Not Applicable



Execution & Plan





Validation Plan

Paragraph #	Test Name	Success Criteria	Methodology	Status	Responsible Engineer(s)
3.1	Tile Size Constraint	The final GDSII layout must remain within the 2-tile maximum area allowed for TinyTapeout.	Run final Place and Route in OpenROAD, verify bounding box and padframe placement using the layout area report.	Untested	Full Team
4.1	DRC	Zero DRC violations (including antenna) in Magic and KLayout using the Sky130A/TinyTapeout rule decks.	Run Magic and KLayout DRC on the final GDS and iterate fixes until zero violations.	Untested	Full Team
4.2	Operating Temperature Range	Design must remain functionally valid across process and temperature corners (-55°C to +125°C).	Run timing and power analysis using PDK corner simulations (TT, SS, FF) to ensure performance remains stable within range.	Untested	Talha Ibrahim, Kevin Umanzor
5.1	LVS	Netgen reports "LVS clean" with 0 opens/shorts and all devices/nets/pins matched.	Extract the layout to SPICE and compare against the synthesized gate-level netlist in Netgen with the sky130A setup.	Untested	Full Team
5.2	Primary Input Power	Active power below 200 μ W and lower during idle conditions.	Estimate and verify power through post-layout power analysis and simulation reports in OpenROAD.	Untested	Talha Ibrahim, Kevin Umanzor
5.3	Signal Interfaces	All interfaces (I ² C, SPI, USB) respond correctly at specified logic levels and timings.	Run RTL and gate-level testbenches to verify correct data transfer, logic levels, and signal timing for all interfaces.	Untested	Githin Johny, Giovanni Giagnacovo
5.4	User Control Interface	The reset mechanism returns the CPU to a known initial state.	Run RTL simulations for both synchronous and asynchronous resets to confirm the PC and register file reset correctly.	Untested	Giovanni Giagnacovo, Talha Ibrahim
5.5	Clock Interface	The design accepts a 10 MHz external clock and meets all setup and hold timing constraints.	Run Static Timing Analysis (STA) and post-layout timing simulations in OpenROAD to verify clock stability and timing closure.	Untested	Kevin Umanzor, Githin Johny
6.1	Fetch	The Fetch stage retrieves correct instructions and updates the program counter properly.	Create directed and random test programs in RTL simulation to verify instruction fetch and PC increment behavior.	Tested	Kevin Umanzor
6.2	Decode	The Decode stage correctly generates control signals from instruction fields.	Run RTL testbenches for all supported instructions to confirm control signal accuracy and complete coverage.	Tested	Githin Johny
6.3	Execute	The ALU produces correct results for all arithmetic and logic operations.	Run RTL and gate-level simulations for arithmetic, logical, and branch operations with directed and random test vectors.	Tested	Talha Ibrahim
6.4	Memory/Write Back	The memory and write-back stages perform correct data reads, writes, and register updates.	Run RTL and gate-level simulations with memory models to verify correct load/store behavior and data integrity.	Tested	Giovanni Giagnacovo
	Full System Simulation	The full 16-bit RISC-V design executes test programs correctly through all pipeline stages.	Integrate all modules into a top-level RTL and post-layout simulation, verifying instruction execution, output correctness, and timing.	Untested	Full Team
	Full System Demo	Execute a test program on each corresponding part on the chip measuring correct output for all cases; along with measured power and basic I/O pass	Integrate tests for top level regression and postlayout simulation capturing outputs.	Untested	Full Team



Dwight Look College of

ENGINEERING
TEXAS A&M UNIVERSITY

Thank you!!